

INPUT: Texto en SFML

Aquí tienes el ejemplo básico para mostrar texto en SFML (C++) 👉

 Código mínimo para dibujar texto en SFML

```
#include <SFML/Graphics.hpp>
int main(){
    sf::RenderWindow window(sf::VideoMode(800, 600), "Texto en SFML");

    // 1. Cargar la fuente
    sf::Font font;
    if (!font.loadFromFile("arial.ttf"))
    {
        return -1; // error si no encuentra la fuente
    }

    // 2. Crear el texto
    sf::Text text;
    text.setFont(font);
    text.setString("Hola mundo!");
    text.setCharacterSize(40); // tamaño
    text.setFillColor(sf::Color::White);
    text.setPosition(100.f, 100.f);

    while (window.isOpen())
    {
        sf::Event event;
        while (window.pollEvent(event))
        {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        window.clear();

        // 3. Dibujar el texto
        window.draw(text);
```

```
    window.display();  
}  
  
return 0;  
}
```

⚠ Cosas importantes

La fuente (.ttf) es obligatoria

SFML no tiene fuentes por defecto → tienes que poner un archivo como arial.ttf en tu proyecto.

Ruta típica:

```
font.loadFromFile("assets/fonts/arial.ttf");
```

Si no carga la fuente → no se verá nada

INPUT Como saco el deltaTime del sfml para que el tiempo vaya para todos igual?

Para que el tiempo vaya igual para todos (independencia del frame-rate), se usa un sf::Clock.

```
sf::Clock dtClock; // 1. Creas el reloj fuera del bucle  
  
while (window.isOpen()) {  
    float dt = dtClock.restart().asSeconds(); // 2. Reinicias el reloj cada frame  
    // ...  
    SM.UpdateCurrentScene(dt); // 3. Pasas ese tiempo a las escenas
```

UI

Para la UI se ha basado en el código de los siguientes enlaces:

Botones: <https://en.sfml-dev.org/forums/index.php?topic=18574.0> y

<https://www.sfml-dev.org/tutorials/3.0/window/events/#sfeventgetift>

Input field: <https://stackoverflow.com/questions/54681508/how-can-i-add-a-sort-of-text-box-in-sfml-using-keyboard-input-and-sf-text-to-di>

La IA en este apartado ha sido usada para correcciones, más específicamente en estas 4 líneas del inputField las cuales no acababan de funcionar. Anteriormente se encontraban como en el enlace de arriba

```
if (e->unicode == '\b' && !input.empty())  
    input.pop_back();  
else if (e->unicode >= 32 && e->unicode < 128)  
    input += static_cast<char>(e->unicode);
```

Connexión a base de datos

Para el código de conexión y desconexión se ha empleado el código mostrado en las presentaciones de la asignatura, añadiendo el setSchema para establecer la base de

datos que se va a usar.

Preparación Entorno - 19/20

Copiar código test

```
Database_Test (Global Scope)
1  #include <stdlib.h>
2  #include <iostream>
3
4  #include "mysql_connection.h"
5  #include <cppconn/driver.h>
6  #include <cppconn/exception.h>
7
8  #define SERVER "127.0.0.1:3306"
9  #define USERNAME "root"
10 #define PASSWORD ""
11
12 void main()
13 {
14     sql::Driver* driver;
15     sql::Connection* con;
16
17     try
18     {
19         driver = get_driver_instance();
20         con = driver->connect(SERVER, USERNAME, PASSWORD);
21         std::cout << "Conection done." << std::endl;
22
23         con->close();
24         std::cout << "Conection close." << std::endl;
25     }
26     catch (sql::SQLException e)
27     {
28         std::cout << "Could not connect to server. Error message: " << e.what() << std::endl;
29         system("pause");
30         exit(1);
31     }
32
33
34     delete con;
35 }
```

Querys

Creamos statements y agregamos en los statements las variables con setString/Int/Float con la posición del ? y el valor. Igualmente, ha estado extraído de las diapositivas.

SQL Injection - Prepared Statment - Setear campos

```
void UpdatePassword(sql::Connection* con, std::string user, std::string newPassword) {  
    try {  
        //sql::Statement* stmt = con->createStatement();  
        std::string query = "UPDATE users SET password = ? WHERE user = ?";  
        sql::PreparedStatement* stmt = con->prepareStatement(query);  
  
        stmt->setString(1, newPassword);  
        stmt->setString(2, user);  
  
        //int affected_rows = stmt->executeUpdate(query);  
        int affected_rows = stmt->executeUpdate();  
        std::cout << "Number of rows affected: " << affected_rows << std::endl;  
  
        delete stmt;  
    }  
    catch (sql::SQLException& e) {  
        std::cout << "Error while updating password: " << e.what() << std::endl;  
    }  
}
```

Cifrado

El cifrado ha sido extraído del git en donde se ha descargado la librería.

<https://github.com/System-Glitch/SHA256/blob/master>

As a library

```
#include "SHA256.h"

//...

string s = "hello world";
SHA256 sha;
sha.update(s);
std::array<uint8_t, 32> digest = sha.digest();

std::cout << SHA256::toString(digest) << std::endl;
```

Al igual que en el ejemplo, se crea un SHA256, se hace un update pasando el string y un digest para cifrar. Esto se ha delegado en una función que se llama al mandar la query para enviar el texto ya cifrado.

```
std::string DatabaseConnector::HashPassword(const std::string& password)
{
    SHA256 sha;
    sha.update(password);
    return sha.toString(sha.digest());
}
```

```
sql::PreparedStatement* pstmt = con->prepareStatement("CALL LoginPlayer( ?, ? );");
pstmt->setString(1, lrd.username);
pstmt->setString(2, HashPassword(lrd.password));
sql::ResultSet* res = pstmt->executeQuery();
// ...
```

```
sql::PreparedStatement* pstmt = con->prepareStatement("CALL AddPlayer( ?, ? );");
pstmt->setString(1, rrd.username);
pstmt->setString(2, HashPassword(rrd.password));
pstmt->execute();
```

Depuración por consola

También se ha utilizado **Copilot** como apoyo en la creación o revisión de distintos **mensajes de depuración por consola**, especialmente en la parte de lobby y flujo de partida. Esto incluye mensajes al crear o unirse a una sala, al empezar la partida y al mostrar información relevante de red y jugadores.

El objetivo de este uso ha sido mejorar la trazabilidad del programa durante el desarrollo y facilitar la localización de errores sin alterar la lógica principal del proyecto.

Pregunta: Ponme mensaje de texto para saber que funciona parte por parte.

NetworkManager Client

Implementación:

....

```
if (m_isConnected)
{
    std::cout << "Ya conectado al servidor" << std::endl;
    return true;
}
```

...

```
if (status == sf::Socket::Status::Disconnected)
{
    std::cout << "[CLIENT] El servidor ha cerrado la conexion." << std::endl;
    m_isConnected = false;
}
```

...

```
if (listener->listen(port) != sf::Socket::Status::Done) {
    std::cerr << "[CLIENT-P2P] Error al crear listener P2P en puerto " << port <<
std::endl;
    return false;
}
```

```

}

listener->setBlocking(false);

std::cout << "[CLIENT-P2P] Escuchando conexiones P2P en el puerto " << port <<
std::endl;

return true;
}

...

if (listener->accept(*newSocket) == sf::Socket::Status::Done)
{
    std::cout << "[CLIENT-P2P] Un peer (rival) se ha conectado!" << std::endl;
    ...
}

}

...

if (!address)
{
    std::cerr << "[CLIENT] IP invalida: " << ip << std::endl;
    return;
}

if (newSocket->connect(*address, port) == sf::Socket::Status::Done)
{
    std::cout << "[CLIENT] Conectado al rival " << ip << ":" << port << std::endl;
    ...
}

else
{
    std::cerr << "[CLIENT] Error al conectar con el rival " << ip << ":" << port << std::endl;

```

```
}  
}
```

```
void NetworkManager::SendToAllConnections(sf::Packet& packet)  
{  
    for (auto& sock : m_gameConnections)  
    {  
        if (sock->send(packet) != sf::Socket::Status::Done)  
        {  
            std::cerr << "[CLIENT] Error al enviar paquete P2P." << std::endl;  
        }  
    }  
}  
...
```

```
void NetworkManager::ClearConnections()  
{  
    ...  
    std::cout << "[CLIENT] Conexiones P2P cerradas." << std::endl;  
}
```

```
bool NetworkManager::Connect(const sf::IpAddress& serverIp, unsigned short  
serverPort)  
{  
    if (m_socket.connect(serverIp, serverPort) != sf::Socket::Status::Done)  
    {  
        std::cerr << "[CLIENT] Error al conectar con el servidor." << std::endl;  
    }  
    ...  
}  
...
```

```

    std::cout << "[CLIENT] Conectado al servidor "
...
}

void NetworkManager::CloseConnection()
{
...
    std::cout << "[CLIENT] Conexion cerrada correctamente." << std::endl;
}

void NetworkManager::SendCreateRoomRequest(const std::string& roomId, const
std::string& nickname, unsigned short gamePort)
{
    if (!m_isConnected)
    {
        std::cerr << "[CLIENT] No se puede enviar CREATE_ROOM_REQUEST: no hay
conexion." << std::endl;

        return;
    }
...
    if (m_socket.send(packet) == sf::Socket::Status::Done)
    {
        std::cout << "[CLIENT] CREATE_ROOM_REQUEST enviada. RoomId: " << roomId <<
std::endl;
    }
    else
    {
        std::cerr << "[CLIENT] Error al enviar CREATE_ROOM_REQUEST." << std::endl;
    }
}

```

```

void NetworkManager::SendJoinRoomRequest(const std::string& roomId, const
std::string& nickname, unsigned short gamePort)

{
    if (!m_isConnected)
    {
        std::cerr << "[CLIENT] No se puede enviar JOIN_ROOM_REQUEST: no hay
conexion." << std::endl;

        return;
    }
...
    if (m_socket.send(packet) == sf::Socket::Status::Done)
    {
        std::cout << "[CLIENT] JOIN_ROOM_REQUEST enviada. RoomId: " << roomId <<
std::endl;
    }
    else
    {
        std::cerr << "[CLIENT] Error al enviar JOIN_ROOM_REQUEST." << std::endl;
    }
}
...
void NetworkManager::ProcessPacket(sf::Packet& packet)
{
    ....
    default:
        std::cout << "[CLIENT] Paquete recibido no gestionado." << std::endl;
        break;
    }
}

```

```

...

void NetworkManager::HandleCreateRoomResponse(sf::Packet& packet)
{
...

    std::cout << "[CLIENT] CREATE_ROOM_RESPONSE -> "

        << responseData.message

        << std::endl;

...

}

void NetworkManager::HandleJoinRoomResponse(sf::Packet& packet)
{

    std::cout << "[CLIENT] JOIN_ROOM_RESPONSE -> "

        << responseData.message

        << std::endl;

...

}

void NetworkManager::HandleRoomStatusUpdate(sf::Packet& packet)
{

...

    std::cout << "[CLIENT] ROOM_STATUS_UPDATE -> Sala "

...

void NetworkManager::HandleErrorMessage(sf::Packet& packet)
{

    ErrorMessageData errorData;

    packet >> errorData;


    std::cout << "[CLIENT] ERROR_MESSAGE -> "

        << errorData.message

```

```
<< std::endl;
```

```
...
```

NetworkManager Server

```
bool NetworkManager::Start(unsigned short listenPort)
{
    if (m_listener.listen(listenPort) != sf::Socket::Status::Done) {

        std::cerr << "[SERVER] Error al escuchar en puerto " << listenPort << std::endl; return
        false;

    }

    ...

    std::cout << "[SERVER] Escuchando en puerto " << listenPort << std::endl;
    return true;
}

void NetworkManager::AcceptNewClients()
{
    if (m_listener.accept(*newSocket) == sf::Socket::Status::Done)
    {
        ...
        std::cout << "[SERVER] Nuevo cliente conectado. playerId: "
            ...
    }

}

void NetworkManager::ReceiveClientData() {
    for (int i = 0; i < static_cast(m_clients.size()); ++i) {

        ...
        if (status == sf::Socket::Status::Disconnected)
        {
            std::cout << "[SERVER] Cliente desconectado. playerId: "
                .... }
        }

    }

}

void NetworkManager::ProcessPacket(ConnectedClient& client, sf::Packet& packet)
{
    ...
}
```

```

default:
    std::cout << "[SERVER] Paquete no gestionado recibido de playerId "
        ...
}
}

...

void NetworkManager::TryStartGame(const std::string& roomId) {

    ...
    std::cout << "[SERVER] START_GAME enviado para sala " << roomId << std::endl;

    ...
}

...

void NetworkManager::HandleRankingUpdate(ConnectedClient& client, sf::Packet&
packet) {

    RankingUpdateData updateData; packet >> updateData;

    std::cout << "[SERVER] Recibida actualización de ranking de sala " << ...

}

void NetworkManager::ProcessRankingValidation(const std::string& roomId) {

    ...
    if (same)
    {
        std::cout << "[SERVER] Ranking validado para sala " << roomId << ". Actualizando
        BD..." << std::endl;
        ...
    }
    else if (updates.size() >= 4)
    {
        std::cout << "[SERVER] Discrepancia insalvable en ranking de sala " << roomId << ".
        Anulando." << std::endl;
        pendingRankingUpdates.erase(roomId);
    }

}
}

```


RoomManager

```
19         std::cout << "[SERVER][RoomManager] Sala creada: " << roomId
20         << " | creatorPlayerId: " << creatorPlayerId
21         << std::endl;

55         std::cout << "[SERVER][RoomManager] Player " << playerId
56         << " unido a sala " << roomId
57         << " | " << room->playerIds.size()
58         << "/" << room->maxPlayers
59         << std::endl;

76         std::cout << "[SERVER][RoomManager] Player " << playerId
77         << " eliminado de sala " << room.roomId
78         << std::endl;
79
80         if (room.playerIds.empty())
81         {
82             std::cout << "[SERVER][RoomManager] Sala vacia eliminada: "
83             << room.roomId
84             << std::endl;
85
86             m_rooms.erase(m_rooms.begin() + roomIndex);
87         }

101         std::cout << "[SERVER][RoomManager] Sala eliminada: " << roomId << std::endl;
102         m_rooms.erase(m_rooms.begin() + i);
103         return;

173     for (const Room& room : m_rooms)
174     {
175         std::cout << " Sala: " << room.roomId
176         << " | players: " << room.playerIds.size()
177         << "/" << room.maxPlayers
178         << " | inGame: " << room.inGame
179         << "\n";
180
181         for (int playerId : room.playerIds)
182         {
183             std::cout << "     - playerId: " << playerId << "\n";
184         }
185     }
```

LobbyScene

```
77      std::cout << "[Client] P2P port assigned: " << myPort << std::endl;
```

```
112      std::cout << "[CLIENT] roomId vacio al crear sala" << std::endl;
```

```
120      std::cout << "[CLIENT] CREATE_ROOM_REQUEST enviado para sala " << roomId << std::endl;
```

```
131      std::cout << "[CLIENT] roomId vacio al unirse." << std::endl;
```

```
139      std::cout << "[CLIENT] JOIN_ROOM_REQUEST enviado para sala " << roomId << std::endl;
```

Centrar texto en botones

Implementado:

```
sf::FloatRect textBounds = text->getLocalBounds(); //chatgpt centrar texto  
sf::FloatRect rectBounds = rect.getGlobalBounds(); // mismo " "
```

Pregunta: en el boton como centro el texto?

Respuesta:

Sí. Ahora mismo tu `Button` pone el texto con un offset fijo de `+5, +5`, por eso no queda centrado.

 Button

Hazlo así:

En `Button::SetText`

```
</> C++  
  
void Button::SetText(std::string newText)  
{  
    if (!text.has_value()) return;  
  
    text->setString(newText);  
  
    sf::FloatRect textBounds = text->getLocalBounds();  
    sf::FloatRect rectBounds = rect.getGlobalBounds();  
  
    text->setOrigin({  
        textBounds.position.x + textBounds.size.x / 2.f,  
        textBounds.position.y + textBounds.size.y / 2.f  
    });  
  
    text->setPosition({  
        rectBounds.position.x + rectBounds.size.x / 2.f,  
        rectBounds.position.y + rectBounds.size.y / 2.f  
    });  
}
```

Y en el constructor del botón

deja esto:

↗ C++



```
Button::Button(float xPos, float yPos, float width, float height, sf::Font& font)
{
    rect.setPosition({ xPos, yPos });
    rect.setSize({ width, height });

    text.emplace(font);
    text->setCharacterSize(18);
    text->setFillColor(sf::Color::Black);
}
```

O sea: quita esta línea:

↗ C++



```
text->setPosition({ xPos + 5, yPos + 5 });
```

porque esa es la que lo descoloca.  Button

Por qué funciona

- `rect.getGlobalBounds()` te da el centro del botón.
- `text->getLocalBounds()` te da el tamaño real del texto.
- con `setOrigin(...)` pones el origen del texto en su centro.
- luego con `setPosition(...)` lo colocas en el centro del rectángulo.

Si quieres, también te paso cómo centrarlo **horizontalmente solo** y dejarlo un poco más alto o más bajo verticalmente para que visualmente quede mejor.

Fuentes

Conclusión

En resumen, en el trabajo la IA se ha usado como **apoyo técnico puntual** para:

- Resolver errores concretos
- Mejorar mensajes de depuración
- Ajustar el centrado visual de botones
- Pequeño código de ayuda
- Detectar y solucionar errores de configuración de proyecto.